Your subscription game, connected

A visual setup guide for the existing Unity project and its rebuilt PHP backend

1.0 - 28 October 2026



YOUR ROUTE

Keep your original Unity scenes. Install the PHP backend, connect your services, and verify the complete account-to-subscription flow.

Includes illustrated steps, copyable commands, setup checkpoints and troubleshooting. Hosting/Stripe diagrams are clearly marked as illustrations.

START HERE

Follow this setup route

For the game owner or template purchaser setting up the system. You do not need to redesign the Unity scenes.

01	02	03	04
Prepare	Install	Connect	Verify

01 Prepare

Have an HTTPS host, a database, SMTP credentials, a Stripe test account and Unity ready.

02 Install the server

Expose only the public folder. Configure the database, game catalog and private environment file.

03 Connect services

Test email, configure Stripe and schedule the worker commands.

04 Connect Unity

Set the existing SubscriptionSettings asset, then test registration and a test-mode subscription.

CHECKPOINT

Use the contents on the next page to return to the step you need. Keep the backend URL and Game Identifier consistent throughout.

NAVIGATION

Find your next step

Clickable contents. Each chapter ends with a checkpoint or an expected result.

Follow this setup route 2

Gather these before starting 4

What connects to what 5

Upload the right folder 6

Create a fresh database 7

Fill in the private .env file 8

Install dependencies and tables 9

Give the server your game and plans 10

Make verification email work 11

Create recurring test prices 12

Connect management and payment updates 13

Schedule the maintenance commands 14

Connect the existing settings asset 15

Test registration and verification 16

Complete a test subscription 17

Find the failed checkpoint 18

Before you switch to live 19

Commands and verification record 20

Gather these before starting

The PHP files rebuild the service. They cannot recover the expired server's lost database or old user passwords.

You need	What to check
Unity	Use the version in ProjectVersion.txt: 2022.3.62f3 for this project.
Web hosting	HTTPS, a supported PHP release compatible with PHP 8.2+ code, PDO MySQL, cURL, OpenSSL, mbstring and Argon2id.
Database	A supported MySQL/MariaDB server, a database name, and a database user/password.
Server access	Composer and a command line or deployment workflow that can run migrations and scheduled PHP commands.
Email	An SMTP account that permits your chosen sender address.
Stripe	A test-mode account, recurring prices, Customer Portal configuration and webhook signing secret.

NOTE

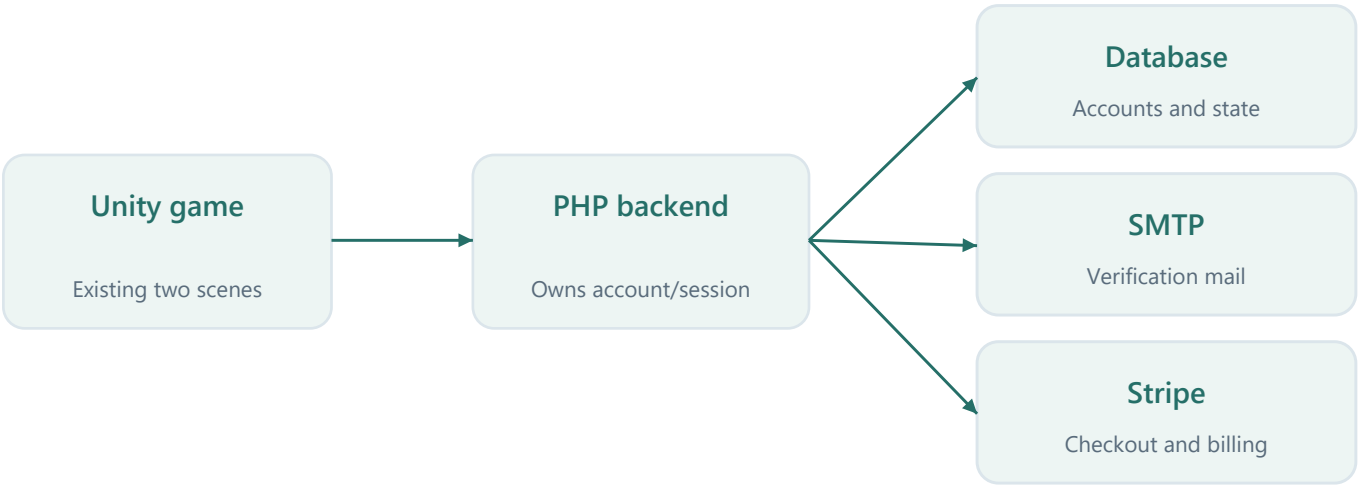
Tested locally with PHP 8.2.12 and MariaDB 10.4.32 in an isolated fixture. Those installed versions are a test record, not a production hosting recommendation. Choose currently supported hosting versions.

CHECKPOINT

Your host must let you expose only public and schedule the worker commands. If it cannot, use a hosting arrangement that can before uploading secrets.

What connects to what

Unity is the player-facing app. Your server owns account identity, email delivery and payment verification.



01 Unity to PHP

Registration and login use the existing PHP filenames. Private calls then carry a session credential.

02 PHP to services

The server reads its private database and uses SMTP and Stripe with server-only credentials.

03 Stripe to PHP

Signed events are received, queued and reconciled. Returning from checkout alone does not unlock a plan.

CHECKPOINT

Keep database, SMTP and Stripe secrets on the server. Unity receives a temporary account session, not your server credentials.

Upload the right folder

Use Backend/SubscriptionServer as the server project. Only its public directory is reachable by a browser.

PRIVATE: SubscriptionServer/

`.env | src/ | config/ | vendor/`

`database/ | bin/ | storage/`

PUBLIC: public/

Your HTTPS URL maps here; PHP endpoints and assets only

01 Choose the public address

Example: https://example.com/subscriptions. That is the URL you will also enter in Unity.

02 Place private files outside the document root

Keep src, config, vendor, bin, database, storage and .env private. Do not upload the whole project into a publicly downloadable directory.

03 Map the URL to public

For a subfolder install, configure an alias or the host's document-root mapping from /subscriptions to the server's public directory. If your host cannot do that safely, use a dedicated subdomain/document root.

CHECKPOINT

A browser should reach public/health.php through your chosen URL. It must not download .env, config/games.php, database files or vendor source.

Create a fresh database

Use your host's database panel or an authorized database administrator. This is a clean installation, not an import of old customers.

Private value	Example, not a credential
DB_HOST / DB_PORT	127.0.0.1 / 3306, or your host's values
DB_DATABASE	subscription_game
DB_USERNAME / DB_PASSWORD	Your private application database account

```
CREATE DATABASE subscription_game
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

Illustrative administrator command. Use your hosting panel if it manages database creation and permissions.

01 Create the database

Example name: subscription_game. Enable utf8mb4 and use an InnoDB-compatible database.

02 Create a database user

Grant the rights needed for migrations during setup. For normal operation, use a user with the required SELECT, INSERT, UPDATE and DELETE rights.

03 Record connection values privately

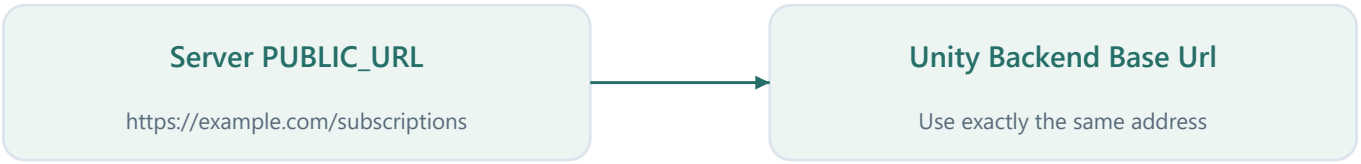
You need host, port, database, username and password. The database host may differ from the website host.

CHECKPOINT

You can connect using the database account intended for setup. Do not use the automated test database command on production data.

Fill in the private .env file

Copy .env.example to .env inside SubscriptionServer. Keep the copy outside public and never commit populated secrets.




```
APP_ENV=development
PUBLIC_URL=https://example.com/subscriptions
BILLING_NAMESPACE=your-generated-installation-id
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=subscription_game
DB_USERNAME=your_database_user
DB_PASSWORD="your-private-database-password"
```

01 Set PUBLIC_URL

Use the final HTTPS origin and folder, without a PHP filename. No /public suffix is needed when the document root is mapped correctly.

02 Generate the installation identifier

Run php bin/generateInstanceId.php after Composer install, then copy its output into BILLING_NAMESPACE. Keep it stable for this installation and use a different value for another deployment.

03 Keep test and live separate

Start with development and Stripe test credentials. Use separate databases and configuration for production.

CHECKPOINT

Replace every example value you use. Do not paste a Stripe secret, SMTP password or database password into Unity.

Install dependencies and tables

Run these commands from Backend/SubscriptionServer, or the matching directory on your server.

```
composer install --no-dev --optimize-autoloader
php bin/generateInstanceId.php
php bin/migrate.php
```

01 Install the locked dependencies

Composer installs Stripe's PHP SDK, PHPMailer and environment-file support from composer.lock. PHP also needs ZIP support or an extraction tool for Composer archives.

02 Save the generated identifier

Copy the non-secret identifier into BILLING_NAMESPACE in .env. Keep .env private.

03 Apply the migrations

Expected result: Database migrations complete. The migration script creates missing tables and records applied migration versions.

NOTE

If PHP or Composer is not available through your host's command line, ask the host for its supported deployment workflow. This guide does not claim an untested upload-only workaround.

CHECKPOINT

Migrations finish without errors. A second run is safe and reports completion. Do not change migration SQL against real data without a backup and migration review.

Give the server your game and plans

Copy config/games.example.php to config/games.php. The private catalog connects a game key to allowed Stripe prices.

```
'Your Game' => [
  'name' => 'My Game',
  'portal_configuration' => 'bpc_your_test_config',
  'plans' => [
    'monthly' => [
      'name' => 'Monthly',
      'description' => 'Premium access, billed monthly.',
      'price_id' => 'price_your_recurring_test_price',
      'display_price' => 'Your actual amount / month',
      'active' => true,
    ],
  ],
],
```

01 Match the game key

The outer key, here Your Game, must exactly match Unity's Game Identifier, including spaces and case.

02 Map only your allowed prices

Use real recurring test price IDs from Stripe. Display text, amount, currency and renewal wording must match the actual checkout.

03 Seed the catalog

Run php bin/seed.php. Expected result: Game catalog seeded; retired price mappings retained.

CHECKPOINT

Do not replace the price ID on an existing plan key. Add a new plan key for a new price; omit or disable old offerings while retaining mappings for existing subscribers.

Make verification email work

Use development mail logging first, then a real SMTP account. Registration queues verification mail; the mail worker delivers it.

Setting	What to enter
MAIL_TRANSPORT	log for local development; smtp for real delivery
MAIL_FROM / MAIL_FROM_NAME	An authorized sender address and game name
SMTP_HOST / SMTP_PORT	Your email provider's SMTP server and port
SMTP_ENCRYPTION	Usually tls for port 587; follow your provider
SMTP_USERNAME / SMTP_PASSWORD	Your private SMTP credentials

```
php bin/processMail.php
```

01 Try the worker

With log transport, read the private storage/mail.log for a development verification link. Never expose that log through the website.

02 Switch to SMTP

Set MAIL_TRANSPORT=smtp and configure your provider. Use a test mailbox and run the worker again.

03 Verify the account

Open the link within one hour. Expected page: Email verified. Return to Unity and refresh the profile.

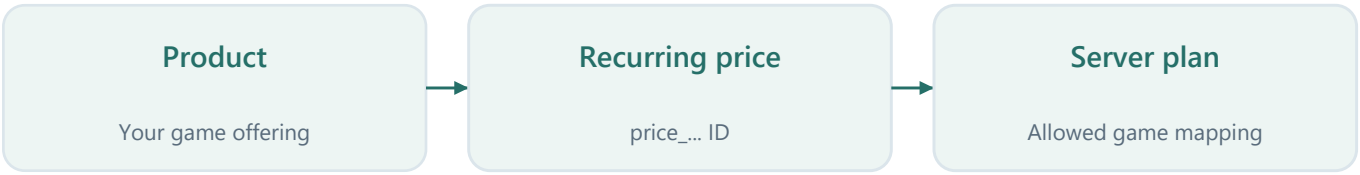
CHECKPOINT

The worker reports sent: 1 or more, and the test mailbox receives the link. Production configuration rejects development mail logging. See the troubleshooting chapter for delivery failures.

Create recurring test prices

Dashboard navigation can change. The diagram below is illustrative; use the linked official guide for the current Stripe screens.

ILLUSTRATION - not a Stripe Dashboard screenshot



01 Stay in the test environment

Create or select the product for your game. Add the recurring prices you want to offer, such as monthly and yearly.

02 Copy each price ID

Put price_... into the matching entry in config/games.php. Do not use a product ID in the price_id field.

03 Configure the private API key

Put your test secret key in STRIPE_SECRET_KEY in .env. It must never be placed in the Unity Inspector.

04 Seed the updated catalog

Run php bin/seed.php and confirm the displayed price/renewal wording matches the hosted checkout.

Official Stripe Checkout guide

Official Stripe testing guide

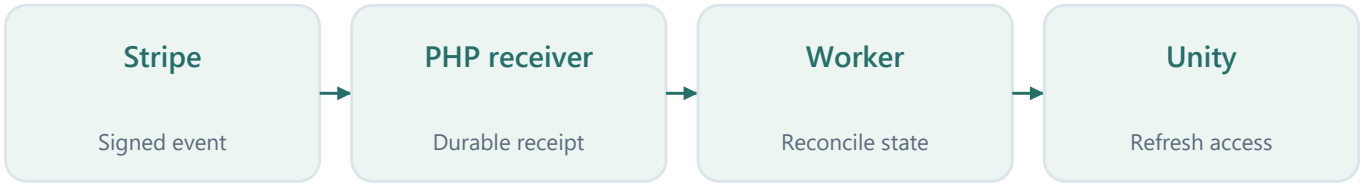
CHECKPOINT

The catalog contains actual recurring test prices, not price_REPLACE values. Do not run a live purchase during this walkthrough.

Connect management and payment updates

The existing Change Plan and Manage buttons open Stripe's Customer Portal through an ownership-checked server link.

PAYMENT CONFIRMATION FLOW



01 Configure the Customer Portal

Enable only the products/prices and operations intended for this game. Decide cancellation timing and plan-change/proration behavior deliberately.

02 Save the portal configuration

Use its bpc_... identifier in the game's portal_configuration. Separate configurations are required when multiple games share this deployment.

03 Register the webhook URL

Use your PUBLIC_URL followed by /stripeWebhook.php. Save that endpoint's signing secret as STRIPE_WEBHOOK_SECRET.

04 Select the relevant events

Use checkout completion/async outcome, subscription created/updated/deleted, and invoice paid/payment_failed events listed in the server README.

Official Customer Portal setup

Official webhook setup

CHECKPOINT

Use the signing secret for this exact endpoint or local CLI listener. Do not mix test/live or CLI/Dashboard webhook secrets.

Schedule the maintenance commands

Use your host's scheduler/cron with its absolute PHP executable and the full path to each script. These are CLI commands, not public browser URLs.

Command inside SubscriptionServer	Suggested starting schedule
php bin/processMail.php	Every minute
php bin/processWebhooks.php	Every minute
php bin/reconcileSubscriptions.php	Every minute; processes up to 100 accounts per run
php bin/cleanup.php	Daily

```
* * * * * /usr/bin/php /srv/subscriptions/bin/processMail.php
* * * * * /usr/bin/php /srv/subscriptions/bin/processWebhooks.php
* * * * * /usr/bin/php /srv/subscriptions/bin/reconcileSubscriptions.php
15 3 * * * /usr/bin/php /srv/subscriptions/bin/cleanup.php
```

Illustrative Linux cron entries. Replace both paths with your host's real paths.

01 Check the output

Mail and webhook jobs report counts. Investigate failed counts in private logs; do not publish logs containing verification links.

02 Recover failed reconciliation

Fix the underlying provider/configuration issue, then use processWebhooks.php --retry-failed for events that exhausted retries.

03 Run the configuration check

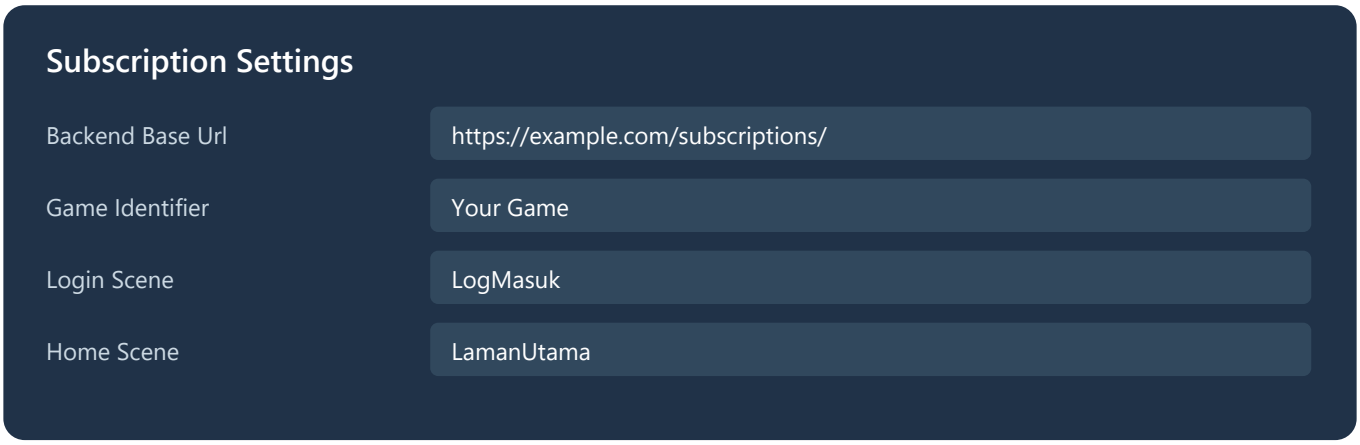
Run php bin/checkConfig.php. Fix every reported issue before testing payments.

CHECKPOINT

Jobs run under the intended server account. Error output is monitored privately. If 100 accounts per pass is too little, adjust worker capacity and monitor reconciliation lag.

Connect the existing settings asset

In Unity, choose Tools > Subscription System > Select Settings. This is the same asset already used by Domains in both original scenes.



Illustrated settings example, based on the delivered SubscriptionSettings fields. The table below lists the values to enter.

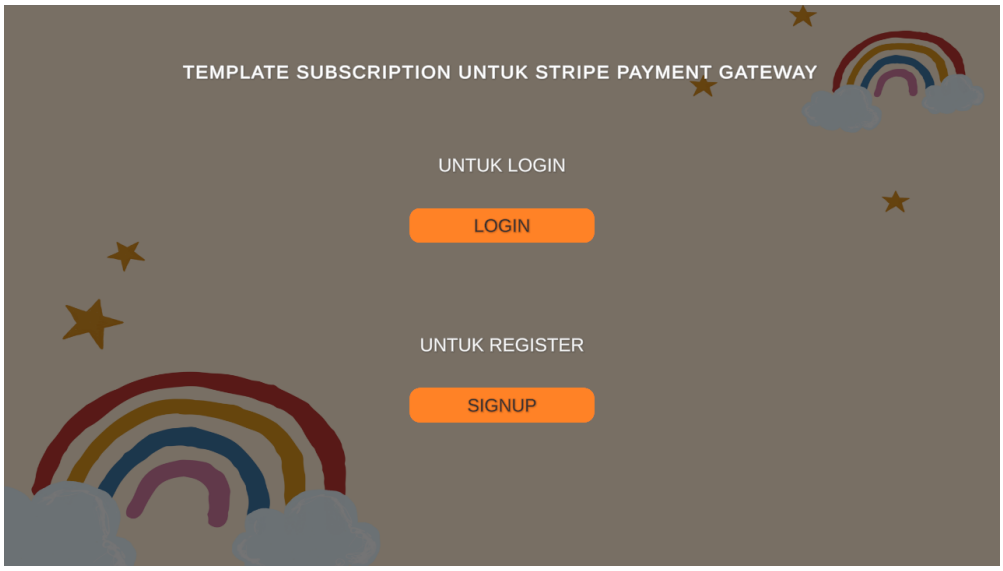
Unity field	Your server value
Backend Base Url	PUBLIC_URL, for example https://example.com/subscriptions
Game Identifier	The exact outer key in config/games.php
Enforce Server Login Status	Keep enabled for the reconstructed backend
Billing Link Endpoint	createBillingLink.php
Other endpoint/page fields	Keep the delivered PHP filenames unless you intentionally change the contract

CHECKPOINT

Open LogMasuk and Validate Current Scene. With a configured valid URL, the setup checks should pass. No server password or Stripe secret belongs in this asset.

Test registration and verification

Start from the original LogMasuk scene. The client now uses an in-memory credential after login; editable PlayerPrefs flags cannot authenticate an account.



Actual existing Unity screen with sample data; no replacement scene is used.

01 Choose SIGNUP

Enter a test name, valid email, Malaysian mobile number and a password of 12-128 bytes (12-128 characters for ASCII text). A successful result confirms account creation, not email delivery.

02 Run/check the mail worker

Open the verification link from the test mailbox or private development log. It expires after one hour and is single use.

03 Choose LOGIN

Sign in with that account. If a previous device session remains active, use the existing authorized logout prompt, then sign in again.

04 Check the profile

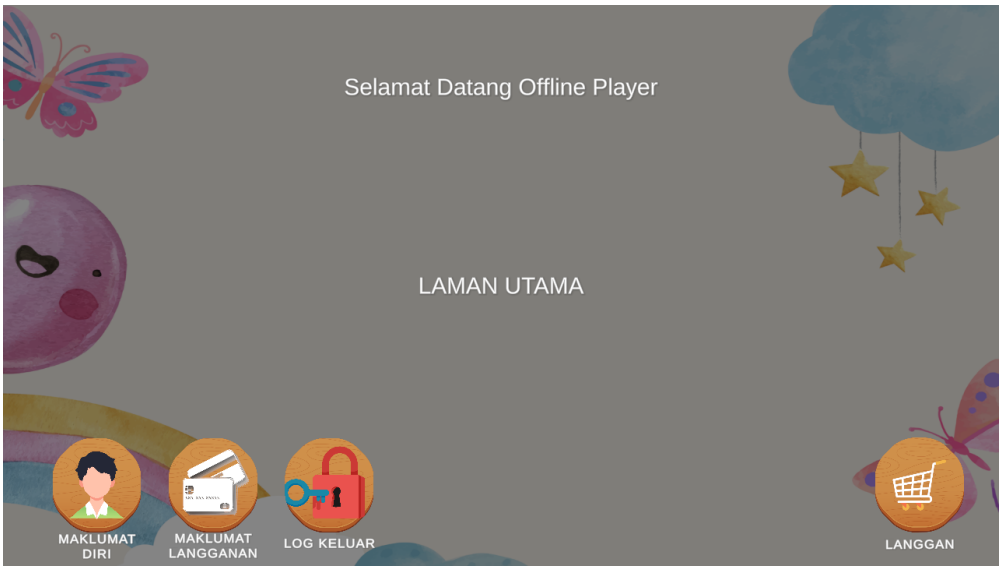
Expected verification state: User Verified after refresh. The server, not the local field, determines this state.

CHECKPOINT

In-memory sign-in intentionally ends after restarting the app. Sign out before closing when practical; otherwise a correct-password device-conflict flow can revoke the old session.

Complete a test subscription

Use only your Stripe test environment and the official test payment methods. The existing LANGGAN button is Subscribe.



Actual existing Unity screen with sample data; no replacement scene is used.

01 Open LANGGAN

A verified signed-in user receives a short-lived billing link. Select a plan and confirm its actual amount/currency on Checkout.

02 Complete the test checkout

Use an official Stripe test payment method. A success page tells you to return to Unity; it does not itself unlock premium content.

03 Process and refresh

Let the webhook worker run. Return to the game and reopen subscription details if needed. Expected status: ACTIVE with a future access-end date.

04 Test management

Use the existing plan-change/management controls. Check their effective dates, any amount due and cancellation policy before confirming.

Stripe test payment methods

CHECKPOINT

Test renewal failure, cancellation at period end, immediate cancellation and expiry. Active access lasts only through the paid period; other provider states do not silently become ACTIVE.

Find the failed checkpoint

Fix the earliest failed checkpoint first. Do not bypass authentication or certificate validation to make a symptom disappear.

Symptom	Check / next action
Blank URL or setup notice	Set Backend Base Url and the matching game identifier in the existing settings asset.
Database/migration failure	Check DB host/port/user permissions and that CLI PHP has PDO MySQL enabled.
Email never arrives	Run processMail.php, inspect failed counts, verify SMTP sender/credentials and spam folder.
401 / session expired	Sign in again. Memory-only credentials do not survive an app restart; revoked sessions cannot be reused.
Other-device prompt	Use the password-authorized logout action, then retry login. Device name is a label, not proof of identity.
Billing link expired	Return to Unity and request a fresh link. Do not bookmark or share ticket URLs.
Webhook signature failure	Check the raw body, signing secret for that endpoint, test/live environment and server clock.
Paid but not ACTIVE	Run webhook/reconciliation jobs. Verify mapped recurring price, period end and provider status; refresh Unity.
Checkout pending after interruption	Retry the same plan. If an old intent needs repair, reconcile provider records before creating another subscription.
Portal offers wrong plans	Correct the game's portal configuration and allowed price catalog before continuing.

CHECKPOINT

When reporting an issue, include the step and non-sensitive error code. Remove passwords, credentials, verification links and billing tickets from screenshots/logs.

Before you switch to live

Passing local tests is not a deployment or a real payment test. Complete this checklist in your own hosting and device environment.

- [ ] Test-mode flow completed
- [ ] SMTP and workers monitored
- [ ] Secrets remain outside public
- [ ] Platform/device checks completed

01 Separate production

Use a production database, SMTP setup, live Stripe credentials and live webhook endpoint. Keep a unique stable installation namespace.

02 Enable live mode explicitly

Set APP_ENV=production and ALLOW_LIVE_PAYMENTS=true only when ready. Use SMTP; development mail logging is blocked.

03 Verify distribution requirements

Choose the billing integration appropriate for your target platform/store before release. This project does not add native store billing.

04 Back up and monitor

Back up private configuration and database, monitor failed mail/webhooks and reconciliation lag, and test a restore procedure.

CHECKPOINT

Do not recreate old paid subscriptions to recover lost users. Historical account/customer recovery needs a separate ownership-verified migration, not an email-only match.

REFERENCE

Commands and verification record

Keep this page with the server README. Commands are run from Backend/SubscriptionServer unless a full path is shown.

Verified locally	Not claimed as verified
Existing Unity project and original scenes	Your hosting panel / public HTTPS deployment
PHP 8.2.12 + isolated MariaDB 10.4.32 fixture	Live SMTP mailbox delivery
Locked Stripe SDK 19.4.1, PHPMailer 7.1.1, dotenv 5.8.0	Real Stripe account/test purchase and Portal configuration
Authentication, ownership, idempotency and event tests with a test provider	Real Android/iOS WebView and store billing review

```
composer install --no-dev --optimize-autoloader
php bin/generateInstanceId.php
php bin/migrate.php
php bin/seed.php
php bin/checkConfig.php
php bin/processMail.php
php bin/processWebhooks.php
php bin/reconcileSubscriptions.php
php bin/cleanup.php
```

Official Stripe webhook reference

PHP supported versions

CHECKPOINT

Guide version 1.0. Screenshots use sample/test data. Hosting and Stripe diagrams are illustrative, not screenshots of your accounts.